SDSU Machine Learning Course (CompE510/EE600/CompE596)

Table of Contents

Programming Assignment: K-Means Clustering 1
========== Part 1: Load Image ========== 1
========== Part 2: Cluster Pixels with K-means ========== 2
========== Part 3: Construct the Compressed Image ========== 4

Programming Assignment: K-Means Clustering

```
Instructions
------------

Dataset comes from:
 https://leafyplace.com/types-of-birds/

This file contains code that helps you get started on the
k-means clustering assignment.

For this part of the assignment, you will need to change some
parts of the code below for various experiments (e.g., changing
the number of clusters).
```

```
% Initialization
clear ; close all; clc
```

========== Part 1: Load Image ==========

```
fprintf('===== Part 1: Load Image ===== \n');

% Instructions: The following code loads and display image bluebird.jped
%               Hint: you can use functions "imread()" and "imshow()"
% ============================================================

% ====================== YOUR CODE HERE ======================
% Step 1: read the image and store it in "img"
img = imread('bluebird.jpeg'); %you need to change this

% Step 2: display the image
imshow(img);

% ============================================================
```

```
fprintf('Program paused. Press enter to continue.\n');
pause;
```

```
===== Part 1: Load Image =====
Program paused. Press enter to continue.
```



========== Part 2: Cluster Pixels with K-means ==========

```
fprintf('===== Part 2: Cluster Pixels with K-means ===== \n');

% Instructions: The following code cluster the data points (pixels) into K
%               clusters. Tune the value of K to see its impact.
%
%               Procedure:
%               First initilize the centroids
%               Then iterate following steps until convergence (iterate for
a certain number of iterations)
%                   a: Assign each pixel into the cluster with centroid
closest to it
%                   b: Update the centroids
% =============================================================


% ====================== YOUR CODE HERE ======================
% Step 1: Configure the number of clusters
K = 20; % you need to change this

% Step 2: scale the pixel values so that they fall within [0,1]
```

```
img_double = im2double(img);


% Step 3: convert the image as a 2d matrix
% Hint: if an image is of dimension (n,m,c), then reshape it
%       to a matrix of dimension (n*m,c)

n = length(img(:,1,1));
m = length(img_double(1,:,1));
c = length(img_double(1,1,:));
img_convert = reshape(img_double,n*m,c);

% Step 4: initilize the cluster centroids

centroids = randperm(length(img_convert),K);

centroids = img_convert(centroids,:);



% Step 5: iterate to cluster the pixels
num_iterations = 50; % change this
SSE = zeros(num_iterations,1); %initialize SSE vector

for iter = 1:num_iterations
% Step 5a: assign each pixel into a cluster
D = pdist2(img_convert,centroids);
[~, clusterLabels] = min(D, [], 2);
SSE(iter) = sum(min(D, [], 2).^2);




% Step 5b: recalculate the centroids
for clusterIndex = 1:K
    assignedPixels = img_convert(clusterLabels == clusterIndex, :);
    if ~isempty(assignedPixels)
        centroids(clusterIndex, :) = mean(assignedPixels, 1);
    end
end







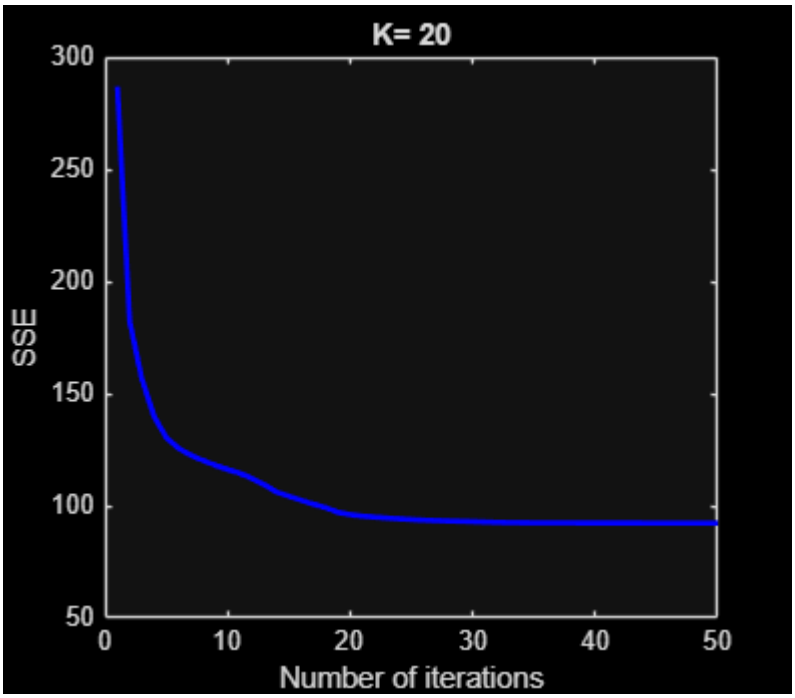
end

% ============================================================
% Step 6: Plot a graph that shows the change of SSE (Sum of Squared Error)
as a way
% to check convergence
figure;
...
```

```
plot(SSE, '-b', 'LineWidth', 2);
xlabel('Number of iterations');
ylabel('SSE');
str = sprintf('K= %d', K);
title(str);

fprintf('Program paused. Press enter to continue.\n');
pause;
```

```
===== Part 2: Cluster Pixels with K-means =====
Program paused. Press enter to continue.
```



========== Part 3: Construct the Compressed Image ==========

```
fprintf('===== Part 3: Construct the Compressed Image ===== \n');

% Instructions: The following code construct and display the compressed
image.
%               Hint: pixels within the same cluster have the same value as
%               the cluster centroid.
%
%
% ====================== YOUR CODE HERE ======================


img_compressed = centroids(clusterLabels,:);
img_compressed = reshape(img_compressed,n,m,c);
```

```
% ============================================================

% store the compressed image
imwrite(img_compressed, 'bluebird_compressed_K5.jpeg');
% display the compressed image
fprintf('Compression is done. \n');
figure;
imshow(img_compressed)
```

```
===== Part 3: Construct the Compressed Image =====
Compression is done.
```



Published with MATLAB® R2026b